ENGINEERING DOCUMENTATION / 02

API Documentation

Contracts, examples and accounting semantics

A developer reference for using every implemented HTTP operation, with precise money, validation, filtering and settlement behavior.

Document reference	Value
Project	Expense Sharing App / Fairshare UI
Edition	Final recruiter submission
Prepared	16 October 2026
Application baseline	72f1182
API / schema	0.1.0 / Alembic head 0002
Currency / default timezone	INR / Asia/Kolkata

Inside this guide

- Conventions and error handling
- Worked trip and optional split modes
- Balances, settlement suggestions and payments
- Search, categorization and analytics
- Complete operation and model reference

Verified Render demo: https://expense-sharing-app-inda.onrender.com/

Source repository: https://github.com/manansethi1205/expense-sharing-app

Live UI and API verified on 16 October 2026. Free hosting can sleep; the free database expires 30 days after creation. Shared synthetic data only.

Examples use synthetic data. IDs shown in worked examples are illustrative; use IDs returned by your own requests.

Contents

1. API conventions 3

2. Validation and error responses 4

3. Worked trip: create and share 5

4. Custom splits and history 6

5. Balances and settlement 7

6. Search and smart categories 9

7. Spending analytics 10

8. Complete endpoint reference 11

9. JSON field dictionary 24

10. Postman and integration notes 32

11. Deployed UI and live verification 33

1. API conventions

Base address and live contract

Live API base: https://expense-sharing-app-inda.onrender.com/api/v1
Interactive contract: https://expense-sharing-app-inda.onrender.com/docs. The deployed UI at the root calls these same financial APIs. Local development uses the addresses below.

```
Application: http://localhost:8000
API prefix:  /api/v1
Swagger UI:  http://localhost:8000/docs
ReDoc:       http://localhost:8000/redoc
OpenAPI:     http://localhost:8000/openapi.json
```

Use Content-Type: application/json for bodies. The service is a single-currency INR demo with no authentication. User/group scope filters do not provide access control. Do not expose a shared instance containing private financial data.

Convention	Meaning
Identity	IDs, not display names, identify entities. Duplicate names are permitted.
Money input	Use decimal strings such as "1200.00". JSON numbers such as 1200 are accepted, but clients should avoid floating-point calculations.
Money output	Amounts are strings with two decimal places. Storage and domain arithmetic use integer paise.
Body validation	Unknown fields are rejected. Body IDs must be positive integers; string IDs are rejected. Text containing a NUL character is rejected.
Expense amount	Greater than 0; at most INR 10,000,000.00; no more than two meaningful decimal places. Custom shares may be zero.
Settlement amount	Greater than 0 and at most INR 92,233,720,368,547,758.07, the positive BIGINT paise range. The current debt may impose a much lower limit.
Timestamps	Use an explicit UTC offset, e.g. 2026-10-16T19:30:00+05:30. occurred_at defaults to now; created_at is generated by the service.
Pagination	Lists return items, total, limit and offset. Default limit=20, maximum=100; offset starts at 0. Search additionally caps offset at 1000.

Typical list envelope

```
{"items":[],"limit":20,"offset":0,"total":0}
```

> **Retries**
>
> Membership PUT is safe to repeat. Creation POST requests have no idempotency key; a network retry may create another record. Verify the previous result before retrying a payment record.

2. Validation and error responses

All handled errors use the same JSON envelope. details contains structured validation information when applicable; clients should branch on code and status rather than parsing the human message.

```
{"error":{"code":"payer_not_member","message":"Payer must be a group member","details":[]}}
```

Status	When it occurs	Typical codes
404	A referenced user/group is absent, or the expense is outside this group.	user_not_found, group_not_found, expense_not_found
422	Malformed body, nonpositive amount, excess precision, invalid custom shares, unknown timezone, bad query/date range.	validation_error, invalid_splits, invalid_timezone, invalid_date_range, invalid_search, invalid_search_types
409	Current state prevents an operation: payer not a member, invalid settlement participants/direction, self-payment or overpayment.	payer_not_member, invalid_settlement, user_not_member, integrity_conflict
503	Readiness cannot reach the database.	database_unavailable

A 422 validation response may identify a location such as body.amount. A 409 settlement failure means the client should read current balances and refresh suggestions. No partial expense shares or partial payment records are committed on failure.

Dates and reporting timezones

Query parameter from is inclusive and to is exclusive. Both are local dates in the selected timezone: from=2026-10-01&to=2026-11-01 covers October. Expense and settlement history use the group's timezone. Analytics accepts a timezone override; a cross-group user report defaults to Asia/Kolkata. A user report restricted to one group uses that group's timezone unless overridden.

A backdated expense still uses the group's member list at recording time. Its occurred_at affects date-based reporting; created_at preserves when the service recorded it.

3. Worked trip: create and share

Example IDs below assume a fresh database. Always use actual IDs returned by the server. The supplied Postman collection captures them automatically and works even when seed data already exists.

Step 1 - create three users

```
POST /api/v1/users
{"name": "Mohit"}
```

Repeat for Rahul and Ankit. Each response is 201 with id, name and created_at. This example uses user IDs 1, 2 and 3 respectively.

Step 2 - create a group and add members

```
POST /api/v1/groups
{"name": "Goa Trip", "timezone": "Asia/Kolkata"}

PUT /api/v1/groups/1/members/1
PUT /api/v1/groups/1/members/2
PUT /api/v1/groups/1/members/3
```

The member calls have no request body. A new membership returns 201. Repeating the same PUT returns 200 without duplicating it.

Step 3 - record the dinner

```
POST /api/v1/groups/1/expenses

{"paid_by":1,"amount":"1200.00","description":"Dinner"}
```

The 201 response is an ExpenseRead object. It includes id, group_id, payer label, amount, currency, description, split_type, category provenance, timestamps and saved shares. The share portion is:

```
{
  "shares": [
              {"user":{"id":1,"name":"Mohit"},"amount":"400.00"},
              {"user":{"id":2,"name":"Rahul"},"amount":"400.00"},
              {"user":{"id":3,"name":"Ankit"},"amount":"400.00"}
            ]
}
```

> **Exact allocation**
>
> The service divides integer paise. INR 100.00 among IDs 1, 2 and 3 becomes 33.34, 33.33 and 33.33. Remainder paise go to ascending user IDs. The saved shares always sum to the original bill; the payer's own share never becomes self-debt.

4. Custom splits and history

Equal splitting is the default required behavior. Unequal allocation is an explicit optional mode rather than a change to that default.

```
POST /api/v1/groups/1/expenses

{
  "paid_by": 1,
  "amount": "1000.00",
  "description": "Activity package",
  "split_type": "custom",
  "splits": [{"user_id":1,"amount":"500.00"},{"user_id":2,"amount":"300.00"},{"user_id":3,"amount":"200.00"}]
}
```

Every current member must appear exactly once, including the payer. A zero share is allowed. Duplicates, outsiders, missing members, negative shares or an incorrect total produce 422. The expense and shares are inserted in one transaction.

Filter and inspect history

```
GET /api/v1/groups/1/expenses?category=food&limit=20&offset=0
GET /api/v1/groups/1/expenses/1
```

History also accepts q, paid_by, from and to. GET returns saved participant shares, not a reconstruction from today's memberships. Adding a fourth member leaves this dinner at three shares.

Category correction

```
PATCH /api/v1/groups/1/expenses/1/category
{"category": "food"}
```

Only category metadata can be updated. Financial amount/payer/share edits, deletion and member removal are deliberately absent to preserve the simple audit trail.

5. Balances and settlement

The examples in this section use only the original 1200.00 dinner, before the separate custom-split example is added.

```
GET /api/v1/groups/1/balances?view=direct
```


```
{
  "group_id": 1,
  "currency": "INR",
  "view": "direct",
  "is_settled": false,
  "balances": [
               {"from_user":{"id":2,"name":"Rahul"},"to_user":{"id":1,"name":"Mohit"},"amount":"400.00"},
               {"from_user":{"id":3,"name":"Ankit"},"to_user":{"id":1,"name":"Mohit"},"amount":"400.00"}
             ],
  "explanation": null
}
```


Net view is more compact

GET /api/v1/groups/1/balances?view=net returns signed positions: Mohit +800.00, Rahul -400.00 and Ankit -400.00. Positive means receivable; negative means payable. Net positions sum to zero. The payer's 400.00 consumption is already included.

```
net = expenses_paid - allocated_shares
      + settlements_sent - settlements_received
```

Preview before paying

```
GET /api/v1/groups/1/settlement-suggestions?mode=simplified
```

The PlanRead response contains mode, strategy, objective, optimality_guaranteed, transaction_count, transfers and an explanation. A preview is advisory: it neither stores a plan nor transfers money. Transfer labels include both user IDs and names.

Mode / strategy	Behavior and tradeoff
simplified / exact	Within the configured threshold (default 16 nonzero net positions), a zero-sum subset dynamic program guarantees the minimum transfer count. Cost grows exponentially: O(n * 2^n) time and O(2^n) space.
simplified / greedy	For larger groups, match largest debtors and creditors using heaps in O(n log n). At most n-1 transfers; no minimum-count guarantee.
direct / direct	Remove circulation deterministically, preserving net positions and existing debtor-creditor relationships. The returned actionable graph is acyclic; minimum count is not promised.

Record the external payment

```
POST /api/v1/groups/1/settlements

{"from_user_id":2,"to_user_id":1,"amount":"400.00","mode":"simplified","note":"Paid via UPI"}
```

A 201 SettlementRead confirms recording. Rahul's net position is now zero; Ankit still owes Mohit 400.00. Record Ankit's payment similarly and the group's is_settled becomes true. The service only records cash/UPI/bank payments made outside the application.

> **Current-state validation**
>
> direct payments cannot exceed the current pair debt. simplified payments require a net debtor sending to a net creditor and cannot exceed either position. The server locks the group, recomputes balances and commits atomically, so two concurrent requests cannot both spend the same remaining debt.

Raw direct history may retain a zero-net cycle after simplified settlements. That is an accounting representation, not a new amount payable. Use is_settled and the cycle-free settlement-suggestions endpoint for action. Do not blindly record raw balance edges as payments.

6. Search and smart categories

```
GET /api/v1/search?q=goa&types=groups,expenses
GET /api/v1/search?q=spending%20analytics&types=features
GET /api/v1/search?q=dinner&group_id=1&user_id=2
```

Search returns a paginated list of SearchHit objects: type, id, label, match_field and api_path. It matches case-insensitive substrings and escapes SQL wildcard characters, so % and _ are literal search text. Feature aliases support discovering API capabilities by everyday phrases.

group_id and user_id narrow data results and intersect when combined. User-scoped discovery includes the user and peers from their groups; feature results remain global. These filters are not security boundaries. Search uses straightforward SQL substring matching, suitable for assignment-scale data rather than a full search engine.

```
GET /api/v1/categories
```

Category	Examples / interpretation
travel	Auto, cab, bike transport, taxi, flight and bus tickets.
food	Meals, dinner and contextual hotel dinner.
accommodation	Hotel bookings and lodging.
activities	Activities and entertainment expenses.
shopping	Purchases, including bike purchase rather than bike travel.
other	No confident rule or conflicting unrelated contexts, e.g. hotel dinner and flight.

Rules v2 normalize text, consume matched phrases and examine remaining context. Generic ticket/tickets is weak evidence. category_source records manual, rule or fallback; category_rule preserves the explanation/version where applicable. Supplying category on creation or using the PATCH endpoint overrides inference. Old records are not silently reclassified when rules change.

> **Product choice**
>
> Search reduces navigation effort; transparent categorization makes analytics useful without an external model, API key or unpredictable classification cost. Ambiguity intentionally falls back to other so the user can correct it.

7. Spending analytics

```
GET /api/v1/users/1/analytics?from=2026-10-01&to=2026-11-01
GET /api/v1/groups/1/analytics?timezone=Asia%2FKolkata
GET /api/v1/users/1/analytics/expenses?sort=amount_desc
```

Reports answer two distinct questions: what was consumed, and what cash moved? In the dinner-only example, Mohit has spending=400.00 and paid=1200.00. Rahul has spending=400.00 and paid=0.00 before repayment. The group has spending=1200.00, not 3600.00.

Field	Definition
totals.spending	Personal saved shares, or full group expenses once each.
totals.paid	Expense cash paid by the user, or all expense cash paid in the group.
settlements_sent / received	Recorded external repayments in the selected period; separate from consumption.
net_cash_outflow	paid + settlements_sent - settlements_received.
categories / months	Amount and percentage breakdowns of spending. Monthly is the supported bucket.
groups / members	Group spending breakdown; member allocation breakdown is provided for group reports.
largest_expenses	Up to five named rows with group, payer, category, timestamp, personal/group amount and original bill amount.
drilldown_path	A filter-preserving API path for the full paginated underlying expense list.
metric_definitions	In-response explanations of the report's measures.

Report rows are AnalyticsExpenseRead objects. In a user report amount is the user's share; expense_amount is the whole bill. In a group report amount is the full bill. Category filtering is available on drill-downs; sorting supports amount_desc or date_desc with stable tie breaking.

The totals and breakdowns use a single PostgreSQL read-only repeatable-read snapshot. SQL aggregation avoids loading the entire expense history into Python. Writers are not blocked by the reporting transaction. Percentage values use two decimal places; an empty report yields zero values rather than division errors.

> **Period versus lifetime**
>
> A payment in November for an October expense appears in November cash flow. Date-limited analytics is not an outstanding-debt calculation. Balances and settlement suggestions use the full group ledger.

8. Complete endpoint reference

The following inventory is generated from this revision's OpenAPI document. Path parameters are required. Query defaults and limits are shown where declared. Request/response model names refer to the field dictionary in section 9. The /api/v1 prefix is included explicitly to avoid ambiguity.

POST /api/v1/users

Create a user. Names are trimmed, nonblank and at most 100 characters. Duplicate names are allowed; use the returned ID as identity.

JSON body: UserCreate

Status	Response model	Description
201	UserRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/users

List users, optionally matching q in the name. Results use the standard pagination envelope.

Parameter	Location	Type / constraints
q	query; optional	string | null
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_UserRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/users/{user_id}

Fetch one user by ID; a missing user returns 404.

Parameter	Location	Type / constraints
user_id	path; required	integer

Status	Response model	Description
200	UserRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

POST /api/v1/groups

Create an INR group. timezone is an IANA zone and defaults to Asia/Kolkata. New groups have no members.

JSON body: GroupCreate

Status	Response model	Description
201	GroupRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups

List groups. user_id restricts results to groups containing that user; q matches group names. Filters are discovery filters, not authorization.

Parameter	Location	Type / constraints
user_id	query; optional	integer | null
q	query; optional	string | null
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_GroupRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}

Return group details, timezone, currency and current member_count.

Parameter	Location	Type / constraints
group_id	path; required	integer

Status	Response model	Description
200	GroupRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

PUT /api/v1/groups/{group_id}/members/{user_id}

Add an existing user to an existing group. The URI identifies the membership, so repeating the call is idempotent: 201 when created, 200 when already present. No body is needed.

Parameter	Location	Type / constraints
group_id	path; required	integer
user_id	path; required	integer

Status	Response model	Description
200	MemberRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content
201	MemberRead	Membership created

The runtime also returns 200 when the membership already exists.

GET /api/v1/groups/{group_id}/members

List current members with user IDs, names and joined_at timestamps. This is the member set used for a newly recorded expense.

Parameter	Location	Type / constraints
group_id	path; required	integer
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_MemberRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

POST /api/v1/groups/{group_id}/expenses

Atomically save an expense and its participant shares. Omit split_type for equal splitting. Custom mode must include every current member exactly once. The payer must be a current group member.

Parameter	Location	Type / constraints
group_id	path; required	integer

JSON body: EqualExpense | CustomExpense

Status	Response model	Description
201	ExpenseRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/expenses

List expense history with optional text, category, payer and local-date filters. Each item includes its saved shares; later membership changes never rewrite them.

Parameter	Location	Type / constraints
group_id	path; required	integer
q	query; optional	string | null
category	query; optional	travel, food, accommodation, activities, shopping, other | null
paid_by	query; optional	integer | null
from	query; optional	string (date) | null
to	query; optional	string (date) | null
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_ExpenseRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/expenses/{expense_id}

Fetch a full expense and all shares. An expense outside the supplied group is treated as not found.

Parameter	Location	Type / constraints
group_id	path; required	integer
expense_id	path; required	integer

Status	Response model	Description
200	ExpenseRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

PATCH /api/v1/groups/{group_id}/expenses/{expense_id}/category

Correct a category by sending {"category":"travel"}, for example. The category source becomes manual. No amount, payer or split edits are supported.

Parameter	Location	Type / constraints
group_id	path; required	integer
expense_id	path; required	integer

JSON body: CategoryUpdate

Status	Response model	Description
200	ExpenseRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/balances

Return current lifetime balances. direct is the default and returns debtor-to-creditor transfers; net returns signed positions. Positive net means money to receive. Check is_settled for financial completion.

Parameter	Location	Type / constraints
group_id	path; required	integer
view	query; optional	direct, net; default=direct

Status	Response model	Description
200	BalanceRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/users/{user_id}/balances

Return the user's balances grouped by group; net is the default. Only positions/transfers involving the requested user are included, while each group's settled flag describes the whole group.

Parameter	Location	Type / constraints
user_id	path; required	integer
view	query; optional	direct, net; default=net

Status	Response model	Description
200	UserBalancesRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/settlement-suggestions

Preview transfers without changing data. simplified optimizes net positions, using exact optimization within the configured threshold and greedy matching above it. direct cancels debt circulation while preserving creditor relationships.

Parameter	Location	Type / constraints
group_id	path; required	integer
mode	query; optional	direct, simplified; default=simplified

Status	Response model	Description
200	PlanRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

POST /api/v1/groups/{group_id}/settlements

Record money already paid externally. mode is required. Fresh balances are checked under the group lock; invalid directions, self-payments, outsiders, overpayments and net-settled groups return 409. Partial payments are supported.

Parameter	Location	Type / constraints
group_id	path; required	integer

JSON body: SettlementCreate

Status	Response model	Description
201	SettlementRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/settlements

List recorded payments, optionally filtered by local dates. These are audit records and are separate from expense history.

Parameter	Location	Type / constraints
group_id	path; required	integer
from	query; optional	string (date) | null
to	query; optional	string (date) | null
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_SettlementRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/users/{user_id}/analytics

Report the user's allocated spending and actual cash flows across groups or within group_id. Saved shares define spending; paying a bill for friends does not make its entire amount personal consumption.

Parameter	Location	Type / constraints
user_id	path; required	integer
from	query; optional	string (date) | null
to	query; optional	string (date) | null
group_id	query; optional	integer | null
timezone	query; optional	string | null
bucket	query; optional	literal: month; default=month

Status	Response model	Description
200	AnalyticsRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/analytics

Report group spending, categories, monthly totals and member allocations. An expense is counted once in total spending, while settlement flows are reported separately.

Parameter	Location	Type / constraints
group_id	path; required	integer
from	query; optional	string (date) | null
to	query; optional	string (date) | null
timezone	query; optional	string | null
bucket	query; optional	literal: month; default=month

Status	Response model	Description
200	AnalyticsRead	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/users/{user_id}/analytics/expenses

Paginated evidence behind personal analytics. amount is the user's share and expense_amount is the full bill. Filter by category and dates; choose amount_desc or date_desc ordering.

Parameter	Location	Type / constraints
user_id	path; required	integer
from	query; optional	string (date) | null
to	query; optional	string (date) | null
group_id	query; optional	integer | null
timezone	query; optional	string | null
category	query; optional	travel, food, accommodation, activities, shopping, other | null
sort	query; optional	amount_desc, date_desc; default=amount_desc
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_AnalyticsExpenseRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/groups/{group_id}/analytics/expenses

Paginated evidence behind group analytics. amount and expense_amount both represent the whole expense. Supports category/date filtering and amount_desc or date_desc ordering.

Parameter	Location	Type / constraints
group_id	path; required	integer
from	query; optional	string (date) | null
to	query; optional	string (date) | null
timezone	query; optional	string | null
category	query; optional	travel, food, accommodation, activities, shopping, other | null
sort	query; optional	amount_desc, date_desc; default=amount_desc
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0

Status	Response model	Description
200	Page_AnalyticsExpenseRead_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/search

Search users, groups, expense descriptions and a static feature catalog. q is required. types is a comma-separated subset of users,groups,expenses,features. Exact matches rank before prefixes and substrings, with stable tie breaking.

Parameter	Location	Type / constraints
q	query; required	string; minLength=1, maxLength=100
types	query; optional	string; default=users,groups,expenses,features
group_id	query; optional	integer | null
user_id	query; optional	integer | null
limit	query; optional	integer; default=20, minimum=1, maximum=100
offset	query; optional	integer; default=0, minimum=0, maximum=1000

Status	Response model	Description
200	Page_SearchHit_	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

GET /api/v1/categories

Return available categories and the rule version. Classification is deterministic, explainable and local; it uses no external AI service.

Status	Response model	Description
200	object	Successful Response
404	ErrorRead	Not Found
409	ErrorRead	Conflict
422	ErrorRead	Unprocessable Content

Success object fields: categories (string list), rules_version (string), keywords (category-to-keyword-list object), phrase_overrides (phrase-to-category object), explanation (string).

GET /health/live

Process liveness. Does not contact PostgreSQL. This route is outside /api/v1.

Status	Response model	Description
200	object	Successful Response

Success JSON: {"status":"ok"}

GET /health/ready

Database connectivity readiness using SELECT 1. Returns 503 if unavailable. It does not verify the Alembic revision; migration execution is a separate deployment step.

Status	Response model	Description
200	object	Successful Response
503	ErrorRead	Service Unavailable

Success JSON: {"status":"ready"}

9. JSON field dictionary

Generated from the Pydantic/OpenAPI schemas. 'Required' means the field must be present; a null-capable required response field can still contain null. Read models serialize money as strings. Input money accepts a decimal string or a JSON number. Page models consistently use items, total, limit and offset.

EqualExpense and CustomExpense are selected by split_type; the server injects equal when the discriminator is omitted. The strict-input and financial rules in sections 1-4 also apply beyond the structural fields shown below.

AnalyticsExpenseRead

Field	Required	Type / constraint
expense_id	yes	integer
group	yes	GroupLabel
paid_by	yes	UserLabel
description	yes	string
category	yes	travel, food, accommodation, activities, shopping, other
occurred_at	yes	string (date-time)
amount	yes	string
expense_amount	yes	string

AnalyticsRead

Field	Required	Type / constraint
scope	yes	user, group
id	yes	integer
currency	no	string; default=INR
timezone	yes	string
from_date	yes	string | null
to_date	yes	string | null
totals	yes	Totals
categories	yes	list[Breakdown]
months	yes	list[Breakdown]
groups	yes	list[Breakdown]
members	yes	list[Breakdown]
largest_expenses	yes	list[AnalyticsExpenseRead]
drilldown_path	yes	string
metric_definitions	yes	object

BalanceRead

Field	Required	Type / constraint
group_id	yes	integer
currency	no	string; default=INR
view	yes	direct, net
is_settled	yes	boolean
balances	yes	list[TransferRead] | list[PositionRead]
explanation	no	string | null

Breakdown

Field	Required	Type / constraint
key	yes	string
label	yes	string
amount	yes	string
percentage	yes	string

CategoryUpdate

Field	Required	Type / constraint
category	yes	travel, food, accommodation, activities, shopping, other

CustomExpense

Field	Required	Type / constraint
paid_by	yes	integer; exclusiveMinimum=0.0
amount	yes	number | string
description	yes	string; maxLength=500; minLength=1
category	no	travel, food, accommodation, activities, shopping, other | null
occurred_at	no	string (date-time) | null
split_type	yes	literal: custom
splits	yes	list[Split]; minItems=1

EqualExpense

Field	Required	Type / constraint
paid_by	yes	integer; exclusiveMinimum=0.0
amount	yes	number | string
description	yes	string; maxLength=500; minLength=1
category	no	travel, food, accommodation, activities, shopping, other | null
occurred_at	no	string (date-time) | null
split_type	no	literal: equal; default=equal

ErrorContent

Field	Required	Type / constraint
code	yes	string
message	yes	string
details	no	list[object]

ErrorRead

Field	Required	Type / constraint
error	yes	ErrorContent

ExpenseRead

Field	Required	Type / constraint
id	yes	integer
group_id	yes	integer
paid_by	yes	UserLabel
amount	yes	string
currency	no	string; default=INR
description	yes	string
split_type	yes	string
category	yes	string
category_source	yes	string
category_rule	yes	string | null
occurred_at	yes	string (date-time)
created_at	yes	string (date-time)
shares	yes	list[ShareRead]

GroupCreate

Field	Required	Type / constraint
name	yes	string; maxLength=100; minLength=1
timezone	no	string; default=Asia/Kolkata

GroupLabel

Field	Required	Type / constraint
id	yes	integer
name	yes	string

GroupRead

Field	Required	Type / constraint
id	yes	integer
name	yes	string
currency	yes	string
timezone	yes	string
created_at	yes	string (date-time)
member_count	yes	integer

MemberRead

Field	Required	Type / constraint
id	yes	integer
name	yes	string
group_id	yes	integer
joined_at	yes	string (date-time)

PlanRead

Field	Required	Type / constraint
group_id	yes	integer
currency	no	string; default=INR
mode	yes	direct, simplified
strategy	yes	exact, greedy, direct
objective	yes	minimum_transactions, preserve_original_debts
optimality_guaranteed	yes	boolean
transaction_count	yes	integer
transfers	yes	list[TransferRead]
explanation	yes	string

PositionRead

Field	Required	Type / constraint
user	yes	UserLabel
amount	yes	string

SearchHit

Field	Required	Type / constraint
type	yes	users, groups, expenses, features
id	yes	integer | string
label	yes	string
match_field	yes	string
api_path	yes	string

SettlementCreate

Field	Required	Type / constraint
from_user_id	yes	integer; exclusiveMinimum=0.0
to_user_id	yes	integer; exclusiveMinimum=0.0
amount	yes	number | string
mode	yes	direct, simplified
note	no	string | null
occurred_at	no	string (date-time) | null

SettlementRead

Field	Required	Type / constraint
id	yes	integer
group_id	yes	integer
from_user	yes	UserLabel
to_user	yes	UserLabel
amount	yes	string
currency	no	string; default=INR
mode	yes	string
note	yes	string | null
occurred_at	yes	string (date-time)
created_at	yes	string (date-time)

ShareRead

Field	Required	Type / constraint
user	yes	UserLabel
amount	yes	string

Split

Field	Required	Type / constraint
user_id	yes	integer; exclusiveMinimum=0.0
amount	yes	number | string

Totals

Field	Required	Type / constraint
spending	yes	string
paid	yes	string
settlements_sent	yes	string
settlements_received	yes	string
net_cash_outflow	yes	string

TransferRead

Field	Required	Type / constraint
from_user	yes	UserLabel
to_user	yes	UserLabel
amount	yes	string

UserBalancesRead

Field	Required	Type / constraint
user_id	yes	integer
groups	yes	list[BalanceRead]

UserCreate

Field	Required	Type / constraint
name	yes	string; maxLength=100; minLength=1

UserLabel

Field	Required	Type / constraint
id	yes	integer
name	yes	string

UserRead

Field	Required	Type / constraint
id	yes	integer
name	yes	string
created_at	yes	string (date-time)

10. Postman and integration notes

Import expense-sharing.postman_collection.json and the supplied Local or Render Postman environment. Select Render to use https://expense-sharing-app-inda.onrender.com/api/v1; Local uses http://localhost:8000/api/v1. Environment base_url overrides the collection default. Run from the beginning; scripts capture returned IDs and check expected responses. The 21 requests cover equal/custom expenses, balances, settlements, search, categories and analytics.

The collection is a runnable walkthrough, not an exhaustive negative-test suite. The 25-operation reference above and live OpenAPI include extra retrieval/filter variations and health routes. Unit and integration tests cover invalid inputs, database constraints, concurrency and optimizer guarantees.

Suggested client flow

Create/select a group, load its members, record an expense, show saved shares and category provenance, then display net balances. Before a real external payment, fetch a fresh settlement preview; after the external payment succeeds, record it with the same mode and refresh balances. Treat a 409 as a prompt to refresh state, not to retry blindly.

> **Deliberate scope**
>
> No authentication, invitations, multi-currency conversion, automatic bank/UPI execution, idempotency keys, financial editing or deletion is implemented. These require additional product and integrity decisions. The service focuses on correct sharing, traceable history and explained settlement advice.

11. Deployed UI and live verification

The final submission runs at https://expense-sharing-app-inda.onrender.com/. FastAPI serves the Fairshare UI, its static assets and the documented JSON API from one origin. UI HTML/CSS/JavaScript adds no frontend framework, build step or external CDN dependency.

UI action	API behavior
Choose a trip / person	Group and membership retrieval; individual and group analytics.
Add expense / View split	Expense POST with equal or custom mode; GET reads the saved allocation.
Correct category	PATCH category; category_source becomes manual and reports use the saved result.
Apply report dates	Date filters affect analytics. Outstanding balances remain lifetime values.
Settle up	Fresh direct/simplified suggestions; record a full or partial external payment through the settlement POST.
Fresh demo trip	Ordinary user/group/membership/expense requests create a separate synthetic scenario. Multiple requests are not atomic.
Global search	Search API results open trips in the UI or expose API results/feature documentation.

Two examples with different purposes

The worked API example and Postman runner start with only Dinner 1200.00, producing two 400.00 debts. The deployed sample trip contains Dinner 1200.00, Taxi 600.00 and Hotel 3000.00. It starts with total consumption 4800.00, each share 1600.00, Mohit net +2600.00, Rahul -1000.00 and Ankit -1600.00. Shared data can change; fresh-trip creation recreates the sample.

Verified behavior

The live service passed health, user/group creation, membership, equal allocation, personal analytics, exact suggestions, repayment recording, settled-state verification and search through the existing HTTP smoke script. Local PostgreSQL tests total 87; the application baseline also passed GitHub CI. The smoke trip is separate from the seeded trip.

Failure handling and scope

The client preserves structured JSON validation errors. Plain-text/HTML server or proxy failures are displayed as a useful HTTP-status error instead of a JSON parsing exception. This display behavior does not repair an unavailable database or missing schema. /health/ready verifies database connectivity, while the deployment startup ensures migrations run before service traffic.

> **Reviewer expectations**
>
> The public instance uses free Render hosting, may need a cold start and has a database that expires 30 days after creation. It contains shared synthetic data and no login. Financial POSTs record facts; they do not transfer real money. Use fresh groups for experiments and never enter private financial data.